PYTHON PROGRAMMING

Practical Examination – Complete Answer Booklet

20 questions • Simple Python programs • Expected output • Beginner-friendly explanations

Integrated M.Tech (IMtech)
School of Engineering and Technology
Sanjivani University, Kopargaon

How to use this booklet

• Read the short explanation before typing the program.

• Run each program separately. Save it with a .py extension.

• For file-handling questions, create the mentioned input text file in the same folder as the Python program.

• For plotting and data-analysis questions, install the required libraries: pip install numpy matplotlib pandas.

• Expected outputs are examples. Values or graph appearance may vary slightly depending on input and software version.

Exam reminder: The uploaded question paper asks you to use functions/classes, include a main() or demo section, test valid and invalid inputs, and add titles and axis labels to plots.

Q1. Text Analytics Engine

Simple idea: Uses lowercase, removes punctuation, counts words with a dictionary, and finds palindromes without re or collections.

Python code

```
def analyze_text():
    text = input("Enter paragraph: ").lower().strip()

    clean = ""
    for ch in text:
        if ch.isalnum() or ch.isspace():
            clean += ch
        else:
            clean += " "

    words = clean.split()
    if not words:
        print("Empty input!")
        return

    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1

    top = sorted(freq.items(), key=lambda x: x[1], reverse=True)[:5]
    longest = max(words, key=len)
    avg = sum(len(w) for w in words) / len(words)
    pal = sorted(set(w for w in words if len(w) > 1 and w == w[::-1]))

    print("\n--- TEXT REPORT ---")
    print("Total words:", len(words))
    print("Unique words:", len(freq))
    print("Top words:", top)
    print("Longest word:", longest)
    print("Average word length:", round(avg, 2))
    print("Palindromes:", pal if pal else "None")

if __name__ == "__main__":
    analyze_text()
```

Expected output / result

```
Enter paragraph: Madam and level are good. Madam likes Python!
--- TEXT REPORT ---
Total words: 7
Unique words: 6
Top words: [('madam', 2), ('and', 1), ('level', 1), ('are', 1), ('good', 1)]
Longest word: python
Average word length: 4.14
Palindromes: ['level', 'madam']
```

Q2. Matrix Operations Using Nested Lists

Simple idea: Nested lists are used for matrices. Addition/subtraction need equal dimensions; multiplication needs columns of A equal rows of B.

Python code

```
def add(a, b):
    return [[a[i][j] + b[i][j] for j in range(len(a[0]))]
            for i in range(len(a))]

def subtract(a, b):
    return [[a[i][j] - b[i][j] for j in range(len(a[0]))]
            for i in range(len(a))]

def multiply(a, b):
    if len(a[0]) != len(b):
        print("Multiplication not possible!")
        return None
    result = [[0] * len(b[0]) for _ in range(len(a))]
    for i in range(len(a)):
        for j in range(len(b[0])):
            for k in range(len(b)):
                result[i][j] += a[i][k] * b[k][j]
    return result

def transpose(a):
    return [[a[i][j] for i in range(len(a))] for j in range(len(a[0]))]

def read_matrix(rows, cols):
    matrix = []
    for i in range(rows):
        row = list(map(int, input("Enter row values: ").split()))
        if len(row) != cols:
            raise ValueError("Wrong number of values in row")
        matrix.append(row)
    return matrix

def main():
    try:
        r1, c1 = map(int, input("Rows and columns of A: ").split())
        a = read_matrix(r1, c1)
        r2, c2 = map(int, input("Rows and columns of B: ").split())
        b = read_matrix(r2, c2)

        if (r1, c1) == (r2, c2):
            print("Addition:", add(a, b))
            print("Subtraction:", subtract(a, b))
        else:
            print("Addition/Subtraction not possible!")

        product = multiply(a, b)
        if product is not None:
            print("Multiplication:", product)
        print("Transpose of A:", transpose(a))
    except ValueError as e:
        print("Invalid input:", e)

if __name__ == "__main__":
    main()
```

Expected output / result

```
Rows and columns of A: 2 2
Enter row values: 1 2
Enter row values: 3 4
Rows and columns of B: 2 2
Enter row values: 5 6
Enter row values: 7 8
Addition: [[6, 8], [10, 12]]
Subtraction: [[-4, -4], [-4, -4]]
Multiplication: [[19, 22], [43, 50]]
Transpose of A: [[1, 3], [2, 4]]
```

Q3. Student Record Management

Simple idea: Each roll number is a dictionary key. A student fails if even one subject mark is below 35.

Python code

```
students = {}

def add_student():
    roll = input("Roll number: ")
    if roll in students:
        print("Roll number already exists!")
        return
    name = input("Name: ")
    dept = input("Department: ")
    marks = list(map(float, input("Enter 3 marks: ").split()))
    if len(marks) != 3 or any(m < 0 or m > 100 for m in marks):
        print("Enter exactly 3 marks from 0 to 100.")
        return
    students[roll] = {"name": name, "dept": dept, "marks": marks}
    print("Student added.")

def update_student():
    roll = input("Roll number to update: ")
    if roll not in students:
        print("Student not found!")
        return
    marks = list(map(float, input("Enter 3 new marks: ").split()))
    if len(marks) == 3 and all(0 <= m <= 100 for m in marks):
        students[roll]["marks"] = marks
        print("Updated.")
    else:
        print("Invalid marks.")

def delete_student():
    roll = input("Roll number to delete: ")
    if roll in students:
        del students[roll]
        print("Deleted.")
    else:
        print("Student not found!")

def report():
    if not students:
        print("No records.")
        return
    rows = []
    for roll, s in students.items():
        total = sum(s["marks"])
        per = total / 3
        passed = all(m >= 35 for m in s["marks"])
        grade = "A" if per >= 75 else "B" if per >= 60 else "C" if per >= 50 else "D"
        rows.append((per, roll, s["name"], s["dept"], total, grade,
                     "Pass" if passed else "Fail"))
    rows.sort(reverse=True)
    print("\nRoll Name Total % Grade Status")
    for per, roll, name, dept, total, grade, status in rows:
        print(roll, name, total, round(per, 2), grade, status)
    print("Topper:", rows[0][2])

    depts = {}
    for s in students.values():
        per = sum(s["marks"]) / 3
        depts.setdefault(s["dept"], []).append(per)
    print("Department averages:")
    for d, values in depts.items():
        print(d, round(sum(values) / len(values), 2))

def main():
    while True:
        print("\n1.Add 2.Update 3.Delete 4.Report 5.Exit")
        ch = input("Choice: ")
        if ch == "1": add_student()
        elif ch == "2": update_student()
        elif ch == "3": delete_student()
        elif ch == "4": report()
        elif ch == "5": break
        else: print("Invalid choice!")

if __name__ == "__main__":
    main()
```

Expected output / result

```
1.Add 2.Update 3.Delete 4.Report 5.Exit
Choice: 1
Roll number: 1
Name: Asha
Department: AIML
Enter 3 marks: 80 70 90
```

```
Student added.
Choice: 4
Roll Name Total % Grade Status
1 Asha 240.0 80.0 A Pass
Topper: Asha
Department averages:
AIML 80.0
```

Q4. Recursion Laboratory

Simple idea: All three main tasks use recursion. Binary search returns a zero-based index internally; the demo prints position starting from 1.

Python code

```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(source, "->", target)
        return 1
    moves = hanoi(n-1, source, auxiliary, target)
    print(source, "->", target)
    moves += 1
    moves += hanoi(n-1, auxiliary, target, source)
    return moves

def binary_search(a, key, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if a[mid] == key:
        return mid
    if key < a[mid]:
        return binary_search(a, key, low, mid-1)
    return binary_search(a, key, mid+1, high)

def permutations(text, answer=""):
    if len(text) == 0:
        print(answer)
        return
    for i in range(len(text)):
        if text[i] in text[:i]:
            continue
        permutations(text[:i] + text[i+1:], answer + text[i])

def main():
    n = 3
    print("Tower of Hanoi:")
    total = hanoi(n, "A", "C", "B")
    print("Total moves:", total)

    a = [10, 20, 30, 40, 50]
    print("Position of 40:", binary_search(a, 40, 0, len(a)-1) + 1)
    print("Position of 90:", binary_search(a, 90, 0, len(a)-1))

    print("Unique permutations:")
    permutations("AAB")

if __name__ == "__main__":
    main()
```

Expected output / result

```
Tower of Hanoi:
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
Total moves: 7
Position of 40: 4
Position of 90: -1
Unique permutations:
AAB
ABA
BAA
```

Q5. Course Recommendation Using Sets

Simple idea: Jaccard similarity = size of intersection divided by size of union. Sets automatically avoid duplicate courses.

Python code

```
users = {
    "Asha": {"Python", "Java", "AI"},
    "Ravi": {"Python", "Java", "Web"},
    "Neha": {"Python", "AI", "Data"},
    "Om": {"Java", "Web", "Cloud"},
    "Sara": {"Python", "Data", "Cloud"},
    "Amit": {"Java", "AI", "Cloud"}
}

def similarity(a, b):
    union = a | b
    if not union:
        return 0
    return len(a & b) / len(union)

def recommend(name):
    if name not in users:
        print("Invalid user!")
        return
    scores = []
    for other in users:
        if other != name:
            scores.append((similarity(users[name], users[other]), other))
    scores.sort(reverse=True)

    suggestions = set()
    for score, other in scores:
        suggestions |= users[other] - users[name]
    print("Recommendations:", sorted(suggestions)[:3])

def main():
    name = input("User name: ")
    if name not in users:
        print("Invalid user!")
        return
    print("Courses:", users[name])
    action = input("Add or remove course (a/r): ").lower()
    course = input("Course: ")
    if action == "a":
        users[name].add(course)
    elif action == "r":
        users[name].discard(course)
    print("Updated courses:", users[name])
    recommend(name)

    popular = {}
    for courses in users.values():
        for course in courses:
            popular[course] = popular.get(course, 0) + 1
    print("Most popular course:", max(popular, key=popular.get))

if __name__ == "__main__":
    main()
```

Expected output / result

```
User name: Asha
Courses: {'Java', 'AI', 'Python'}
Add or remove course (a/r): a
Course: SQL
Updated courses: {'Java', 'AI', 'Python', 'SQL'}
Recommendations: ['Cloud', 'Data', 'Web']
Most popular course: Python
```

Q6. Sales Data File Processing

Simple idea: Create sales.txt beside the Python program. Rows with wrong columns, non-numeric values, or negative price/quantity are ignored.

Python code

```
# Create sales.txt in the same folder:
# 1,West,Laptop,50000,2
# 2,East,Phone,15000,3
# 3,West,Mouse,500,4
# 4,East,Tablet,20000,-1

def process_file():
    revenue = {}
    valid = []
    try:
        with open("sales.txt", "r") as file:
            for line in file:
                parts = line.strip().split(",")
                if len(parts) != 5:
                    continue
                try:
                    price = float(parts[3])
                    qty = int(parts[4])
                    if price < 0 or qty < 0:
                        continue
                except ValueError:
                    continue

                region = parts[1]
                amount = price * qty
                revenue[region] = revenue.get(region, 0) + amount
                valid.append(line.strip())

        with open("sales_output.txt", "w") as out:
            out.write("Valid records:\n")
            for row in valid:
                out.write(row + "\n")
            out.write("\nRevenue by region:\n")
            for region, total in revenue.items():
                out.write(region + ": " + str(total) + "\n")

        print("Revenue:", revenue)
        if revenue:
            print("Highest revenue region:", max(revenue, key=revenue.get))
        print("Saved to sales_output.txt")
    except FileNotFoundError:
        print("Error: sales.txt not found!")

if __name__ == "__main__":
    process_file()
```

Expected output / result

```
Revenue: {'West': 102000.0, 'East': 45000.0}
Highest revenue region: West
Saved to sales_output.txt
```

Q7. Bank Transaction Processor

Simple idea: The custom InvalidTransaction exception is raised for invalid amounts, unknown types, and insufficient balance.

Python code

```
class InvalidTransaction(Exception):
    pass

def process_transactions():
    balances = {}
    rejected = 0
    try:
        with open("transactions.txt", "r") as file:
            for line in file:
                try:
                    account, kind, amount = line.strip().split(",")
                    amount = float(amount)
                    if amount <= 0:
                        raise InvalidTransaction("Amount must be positive")

                    balances.setdefault(account, 0)
                    if kind.lower() == "deposit":
                        balances[account] += amount
                    elif kind.lower() == "withdraw":
                        if balances[account] < amount:
                            raise InvalidTransaction("Insufficient balance")
                        balances[account] -= amount
                    else:
                        raise InvalidTransaction("Unknown transaction type")
                except (ValueError, InvalidTransaction) as e:
                    rejected += 1
                    print("Rejected:", line.strip(), "-", e)

        print("Final balances:", balances)
        print("Rejected transactions:", rejected)
    except FileNotFoundError:
        print("transactions.txt not found!")

if __name__ == "__main__":
    process_transactions()

# Example transactions.txt:
# A101,deposit,1000
# A101,withdraw,300
# A101,withdraw,900
# A102,deposit,500
```

Expected output / result

```
Rejected: A101,withdraw,900 - Insufficient balance
Final balances: {'A101': 700.0, 'A102': 500.0}
Rejected transactions: 1
```

Q8. Library Management System

Simple idea: Fine is Rs. 2 for each late day. A member can hold at most three books.

Python code

```
class Book:
    def __init__(self, title):
        self.title = title
        self.issued = False

class Member:
    def __init__(self, name):
        self.name = name
        self.books = []

class Library:
    def issue(self, book, member):
        if book.issued:
            print("Book already issued!")
        elif len(member.books) >= 3:
            print("Member already has 3 books!")
        else:
            book.issued = True
            member.books.append(book)
            print("Issued:", book.title)

    def return_book(self, book, member, late_days=0):
        if book not in member.books:
            print("This member does not have this book.")
            return
        member.books.remove(book)
        book.issued = False
        fine = max(0, late_days) * 2
        print("Returned:", book.title, "Fine: Rs.", fine)

def main():
    library = Library()
    b1, b2 = Book("Python"), Book("Java")
    m1 = Member("Asha")
    library.issue(b1, m1)
    library.issue(b1, m1)
    library.issue(b2, m1)
    library.return_book(b1, m1, 4)
    print("Currently issued:", [b.title for b in [b1, b2] if b.issued])

if __name__ == "__main__":
    main()
```

Expected output / result

```
Issued: Python
Book already issued!
Issued: Java
Returned: Python Fine: Rs. 8
Currently issued: ['Java']
```

Q9. Payroll System Using Polymorphism

Simple idea: Each subclass overrides calculate_pay(). The same method call works for all employee objects.

Python code

```
class Employee:
    def __init__(self, name):
        self.name = name

    def calculate_pay(self):
        return 0

class FullTime(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.salary = monthly_salary

    def calculate_pay(self):
        return self.salary

class PartTime(Employee):
    def __init__(self, name, hours, rate):
        super().__init__(name)
        self.hours = hours
        self.rate = rate

    def calculate_pay(self):
        return self.hours * self.rate

class Intern(Employee):
    def __init__(self, name, stipend):
        super().__init__(name)
        self.stipend = stipend

    def calculate_pay(self):
        return self.stipend

def main():
    staff = [
        FullTime("Asha", 30000),
        PartTime("Ravi", 40, 200),
        Intern("Neha", 8000)
    ]
    total = 0
    for person in staff:
        pay = person.calculate_pay()
        print(person.name, "Salary:", pay)
        total += pay
    print("Average salary:", total / len(staff))

if __name__ == "__main__":
    main()
```

Expected output / result

```
Asha Salary: 30000
Ravi Salary: 8000
Neha Salary: 8000
Average salary: 15333.333333333334
```

Q10. Secure Wallet System

Simple idea: Balance and PIN are private attributes (double underscore). Three consecutive incorrect PIN attempts lock the wallet.

Python code

```
class Wallet:
    def __init__(self, pin):
        self.__balance = 0
        self.__pin = str(pin)
        self.__attempts = 0
        self.__locked = False
        self.history = []

    def verify(self, pin):
        if self.__locked:
            print("Wallet locked!")
            return False
        if str(pin) == self.__pin:
            self.__attempts = 0
            return True
        self.__attempts += 1
        if self.__attempts >= 3:
            self.__locked = True
            print("Wallet locked after 3 wrong PINs!")
        else:
            print("Wrong PIN!")
        return False

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            self.history.append("Deposit: " + str(amount))
        else:
            print("Invalid amount!")

    def withdraw(self, amount, pin):
        if self.verify(pin):
            if 0 < amount <= self.__balance:
                self.__balance -= amount
                self.history.append("Withdraw: " + str(amount))
            else:
                print("Invalid amount or low balance!")

    def change_pin(self, old, new):
        if self.verify(old):
            self.__pin = str(new)
            print("PIN changed.")

    def statement(self):
        print("Balance:", self.__balance)
        print("History:", self.history)

def main():
    w = Wallet("1234")
    w.deposit(1000)
    w.withdraw(200, "1234")
    w.change_pin("1234", "5678")
    w.statement()

if __name__ == "__main__":
    main()
```

Expected output / result

```
PIN changed.
Balance: 800
History: ['Deposit: 1000', 'Withdraw: 200']
```

Q11. Multiple Inheritance and MRO

Simple idea: Cooperative super() passes constructor data through the method resolution order (MRO).

Python code

```
class Person:
    def __init__(self, name, **kwargs):
        self.name = name
        super().__init__(**kwargs)

class Student(Person):
    def __init__(self, course, **kwargs):
        self.course = course
        super().__init__(**kwargs)

class Employee(Person):
    def __init__(self, salary, **kwargs):
        self.salary = salary
        super().__init__(**kwargs)

class TeachingAssistant(Student, Employee):
    def __init__(self, name, course, salary, hours):
        self.hours = hours
        super().__init__(name=name, course=course, salary=salary)

    def stipend(self):
        return self.salary + self.hours * 100

    def show(self):
        print(self.name, self.course, self.salary, self.hours,
              "Stipend:", self.stipend())

def main():
    a = TeachingAssistant("Asha", "Python", 5000, 10)
    b = TeachingAssistant("Ravi", "AI", 6000, 8)
    a.show()
    b.show()
    print("MRO:")
    for cls in TeachingAssistant.mro():
        print(cls.__name__)

if __name__ == "__main__":
    main()
```

Expected output / result

```
Asha Python 5000 10 Stipend: 6000
Ravi AI 6000 8 Stipend: 6800
MRO:
TeachingAssistant
Student
Employee
Person
object
```

Q12. Polynomial Class with Operator Overloading

Simple idea: Coefficients are stored from constant term upward. Special methods overload +, -, *, and function-call ().

Python code

```
class Polynomial:
    def __init__(self, coefficients):
        self.c = coefficients

    def __str__(self):
        terms = []
        for power, value in enumerate(self.c):
            if value != 0:
                terms.append(str(value) + "x^" + str(power))
        return " + ".join(terms[::-1]) if terms else "0"

    def __add__(self, other):
        n = max(len(self.c), len(other.c))
        a = self.c + [0] * (n - len(self.c))
        b = other.c + [0] * (n - len(other.c))
        return Polynomial([a[i] + b[i] for i in range(n)])

    def __sub__(self, other):
        n = max(len(self.c), len(other.c))
        a = self.c + [0] * (n - len(self.c))
        b = other.c + [0] * (n - len(other.c))
        return Polynomial([a[i] - b[i] for i in range(n)])

    def __mul__(self, other):
        result = [0] * (len(self.c) + len(other.c) - 1)
        for i in range(len(self.c)):
            for j in range(len(other.c)):
                result[i+j] += self.c[i] * other.c[j]
        return Polynomial(result)

    def __call__(self, x):
        return sum(value * x**power for power, value in enumerate(self.c))

    def derivative(self):
        if len(self.c) == 1:
            return Polynomial([0])
        return Polynomial([i * self.c[i] for i in range(1, len(self.c))])

def main():
    p = Polynomial([1, 2, 3])   # 1 + 2x + 3x^2
    q = Polynomial([2, 1])      # 2 + x
    r = Polynomial([1, 0, 1])
    print("P =", p)
    print("P + Q =", p + q)
    print("P - Q =", p - q)
    print("P * Q =", p * q)
    print("P(2) =", p(2))
    print("Derivative =", p.derivative())
    print("R =", r)

if __name__ == "__main__":
    main()
```

Expected output / result

```
P = 3x^2 + 2x^1 + 1x^0
P + Q = 3x^2 + 3x^1 + 3x^0
P - Q = 3x^2 + 1x^1 + -1x^0
P * Q = 3x^3 + 7x^2 + 5x^1 + 2x^0
P(2) = 17
Derivative = 6x^1 + 2x^0
R = 1x^2 + 1x^0
```

Q13. Inventory Manager

Simple idea: Inventory stores Product objects using product code as the key. Selling more than available stock is rejected.

Python code

```
class Product:
    def __init__(self, code, name, price, quantity):
        if price < 0 or quantity < 0:
            raise ValueError("Price and quantity cannot be negative")
        self.code = code
        self.name = name
        self.price = price
        self.quantity = quantity

class Inventory:
    def __init__(self):
        self.products = {}

    def add(self, product):
        self.products[product.code] = product

    def restock(self, code, qty):
        if code in self.products and qty > 0:
            self.products[code].quantity += qty
        else:
            print("Invalid product or quantity!")

    def sell(self, code, qty):
        if code not in self.products or qty <= 0:
            print("Invalid sale!")
        elif qty > self.products[code].quantity:
            print("Not enough stock!")
        else:
            self.products[code].quantity -= qty
            print("Sale successful.")

    def report(self, limit):
        value = 0
        for p in self.products.values():
            value += p.price * p.quantity
        print("Total inventory value:", value)
        print("Low stock:")
        for p in self.products.values():
            if p.quantity < limit:
                print(p.name, p.quantity)

    def save(self):
        with open("inventory.txt", "w") as f:
            for p in self.products.values():
                f.write(f"{p.code},{p.name},{p.price},{p.quantity}\n")
        print("Saved to inventory.txt")

def main():
    inv = Inventory()
    inv.add(Product("P1", "Pen", 10, 20))
    inv.add(Product("P2", "Book", 50, 5))
    inv.sell("P1", 3)
    inv.restock("P2", 4)
    inv.report(10)
    inv.save()

if __name__ == "__main__":
    main()
```

Expected output / result

```
Sale successful.
Total inventory value: 1020
Low stock:
Book 9
Saved to inventory.txt
```

Q14. Hospital Appointment System

Simple idea: An appointment is rejected when the same doctor already has that date and time slot.

Python code

```
class Doctor:
    def __init__(self, name):
        self.name = name

class Patient:
    def __init__(self, name):
        self.name = name

class Appointment:
    booked = []

    def __init__(self, doctor, patient, date, time):
        self.doctor = doctor
        self.patient = patient
        self.date = date
        self.time = time

    @classmethod
    def book(cls, doctor, patient, date, time):
        for a in cls.booked:
            if a.doctor.name == doctor.name and a.date == date and a.time == time:
                print("Time slot already booked!")
                return
        cls.booked.append(cls(doctor, patient, date, time))
        print("Appointment booked.")

    @classmethod
    def cancel(cls, patient_name, date, time):
        for a in cls.booked:
            if a.patient.name == patient_name and a.date == date and a.time == time:
                cls.booked.remove(a)
                print("Appointment cancelled.")
                return
        print("Appointment not found!")

    @classmethod
    def show_doctor(cls, doctor_name):
        for a in cls.booked:
            if a.doctor.name == doctor_name:
                print(a.patient.name, a.date, a.time)

def main():
    d = Doctor("Dr. Shah")
    p1 = Patient("Asha")
    p2 = Patient("Ravi")
    Appointment.book(d, p1, "10-11-2026", "10:00")
    Appointment.book(d, p2, "10-11-2026", "10:00")
    Appointment.show_doctor("Dr. Shah")
    Appointment.cancel("Asha", "10-11-2026", "10:00")

if __name__ == "__main__":
    main()
```

Expected output / result

```
Appointment booked.
Time slot already booked!
Asha 10-11-2026 10:00
Appointment cancelled.
```

Q15. Signal Processing with NumPy

Simple idea: Uses a fixed random seed, NumPy z-scores, replacement of outliers by the mean, and a moving-average filter.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

def main():
    np.random.seed(10)
    x = np.linspace(0, 10, 300)
    clean = np.sin(x)
    noisy = clean + np.random.normal(0, 0.2, 300)

    mean = np.mean(noisy)
    sd = np.std(noisy)
    z = np.abs((noisy - mean) / sd)
    outliers = z > 2
    fixed = noisy.copy()
    fixed[outliers] = mean

    kernel = np.ones(5) / 5
    smooth = np.convolve(fixed, kernel, mode="same")

    print("Mean:", round(mean, 3))
    print("Standard deviation:", round(sd, 3))
    print("Outliers:", np.sum(outliers))

    fig, ax = plt.subplots()
    ax.plot(x, noisy, label="Original noisy signal")
    ax.plot(x, smooth, label="Processed signal")
    ax.set_title("Signal Processing")
    ax.set_xlabel("Time")
    ax.set_ylabel("Amplitude")
    ax.legend()
    plt.show()

if __name__ == "__main__":
    main()
```

Expected output / result

```
Mean: (value depends on NumPy version)
Standard deviation: (value depends on NumPy version)
Outliers: (calculated from the generated signal)
A graph appears with original and processed signals.
```

Q16. Business Data Analysis with Pandas

Simple idea: Creates 20 rows, fills the missing Units value with the median, calculates Revenue, groups totals, and exports CSV.

Python code

```
import pandas as pd

def main():
    data = {
        "Date": ["01-10"] * 5 + ["02-10"] * 5 + ["03-10"] * 5 + ["04-10"] * 5,
        "Region": ["West", "East", "North", "West", "East"] * 4,
        "Product": ["Pen", "Book", "Pen", "Bag", "Book"] * 4,
        "Units": [10, 5, 8, 3, 6, 7, 4, 9, 2, 5,
                  6, 8, 3, 4, 7, 5, 6, 10, 3, 8],
        "Price": [10, 50, 10, 200, 50, 10, 50, 10, 200, 50,
                  10, 50, 10, 200, 50, 10, 50, 10, 200, 50]
    }
    df = pd.DataFrame(data)
    df.loc[0, "Units"] = None

    print("First rows:")
    print(df.head())
    print("Missing values:")
    print(df.isnull().sum())

    df["Units"] = df["Units"].fillna(df["Units"].median())
    df["Revenue"] = df["Units"] * df["Price"]

    print("Revenue by region:")
    print(df.groupby("Region")["Revenue"].sum())
    print("Revenue by product:")
    print(df.groupby("Product")["Revenue"].sum())

    best = df.groupby("Product")["Revenue"].sum().idxmax()
    print("Top product:", best)
    df.to_csv("sales_final.csv", index=False)
    print("Saved sales_final.csv")

if __name__ == "__main__":
    main()
```

Expected output / result

```
Revenue by region:
Region
East     1600.0
North     170.0
West     1780.0
Revenue by product:
Product
Bag     2400.0
Book    2050.0
Pen      590.0
Top product: Bag
Saved sales_final.csv
(Exact totals are calculated from the data in the program.)
```

Q17. Multi-Panel Analytical Dashboard

Simple idea: Uses the required object-oriented Matplotlib style: fig, ax = plt.subplots().

Python code

```
import numpy as np
import matplotlib.pyplot as plt

def main():
    months = [1, 2, 3, 4, 5]
    sales = [10, 14, 13, 18, 20]
    marks = [45, 55, 60, 65, 70, 75, 80, 85]
    x = np.array([1, 2, 3, 4, 5])
    y = np.array([2, 4, 5, 4, 6])
    m, c = np.polyfit(x, y, 1)

    fig, ax = plt.subplots(2, 2, figsize=(9, 6))

    ax[0, 0].plot(months, sales, marker="o")
    ax[0, 0].set_title("Sales by Month")
    ax[0, 0].set_xlabel("Month")
    ax[0, 0].set_ylabel("Sales")

    ax[0, 1].hist(marks, bins=5)
    ax[0, 1].axvline(np.mean(marks), label="Mean")
    ax[0, 1].set_title("Marks Distribution")
    ax[0, 1].set_xlabel("Marks")
    ax[0, 1].set_ylabel("Frequency")
    ax[0, 1].legend()

    ax[1, 0].scatter(x, y)
    ax[1, 0].plot(x, m*x+c)
    ax[1, 0].set_title("Scatter and Regression")
    ax[1, 0].set_xlabel("X")
    ax[1, 0].set_ylabel("Y")

    ax[1, 1].bar(["A", "B"], [30, 45], label="Group 1")
    ax[1, 1].bar(["A", "B"], [25, 35], bottom=[30, 45], label="Group 2")
    ax[1, 1].set_title("Grouped Categories")
    ax[1, 1].set_xlabel("Category")
    ax[1, 1].set_ylabel("Value")
    ax[1, 1].legend()

    fig.tight_layout()
    fig.savefig("dashboard.png")
    plt.show()

if __name__ == "__main__":
    main()
```

Expected output / result

```
A window appears with four plots:
1. Sales line plot
2. Marks histogram with mean line
3. Scatter plot with regression line
4. Category bar chart
Saved as dashboard.png
```

Q18. Plot Class Framework

Simple idea: Dashboard calls render() on every object. This is polymorphism; it does not need to check the object's class.

Python code

```
import matplotlib.pyplot as plt

class BasePlot:
    def __init__(self, data, title, xlabel, ylabel):
        self.data = data
        self.title = title
        self.xlabel = xlabel
        self.ylabel = ylabel

class LinePlot(BasePlot):
    def render(self, ax):
        x, y = self.data
        ax.plot(x, y, marker="o")
        self.finish(ax)

    def finish(self, ax):
        ax.set_title(self.title)
        ax.set_xlabel(self.xlabel)
        ax.set_ylabel(self.ylabel)

class BarPlot(BasePlot):
    def render(self, ax):
        labels, values = self.data
        ax.bar(labels, values)
        ax.set_title(self.title)
        ax.set_xlabel(self.xlabel)
        ax.set_ylabel(self.ylabel)

class Dashboard:
    def __init__(self, plots):
        self.plots = plots

    def show(self):
        fig, axes = plt.subplots(1, len(self.plots))
        if len(self.plots) == 1:
            axes = [axes]
        for plot, ax in zip(self.plots, axes):
            plot.render(ax)
        fig.tight_layout()
        fig.savefig("plot_dashboard.png")
        plt.show()

def main():
    p1 = LinePlot(([1, 2, 3], [10, 15, 12]),
                  "Line Plot", "Time", "Value")
    p2 = BarPlot((["A", "B", "C"], [5, 8, 6]),
                 "Bar Plot", "Category", "Value")
    Dashboard([p1, p2]).show()

if __name__ == "__main__":
    main()
```

Expected output / result

```
A figure appears containing:
- One line plot
- One bar plot
Saved as plot_dashboard.png
```

Q19. Student Analytics and Visualisation

Simple idea: Stores each student's three subject marks. The analyzer finds the topper and class average, then draws both plots.

Python code

```
import matplotlib.pyplot as plt

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        return sum(self.marks) / 3

class ClassAnalyzer:
    def __init__(self, students):
        self.students = students

    def analyze(self):
        topper = max(self.students, key=lambda s: s.average())
        class_avg = sum(s.average() for s in self.students) / len(self.students)
        print("Topper:", topper.name)
        print("Class average:", round(class_avg, 2))

        ordered = sorted(self.students, key=lambda s: s.average())
        names = [s.name for s in ordered]
        averages = [s.average() for s in ordered]

        fig, ax = plt.subplots(1, 2, figsize=(10, 4))
        ax[0].barh(names, averages)
        ax[0].set_title("Average Marks")
        ax[0].set_xlabel("Average")

        subject_marks = list(zip(*[s.marks for s in self.students]))
        ax[1].boxplot(subject_marks, labels=["Subject 1", "Subject 2", "Subject 3"])
        ax[1].set_title("Marks Distribution")
        ax[1].set_ylabel("Marks")

        fig.tight_layout()
        fig.savefig("student_analysis.png")
        plt.show()
        print("Observation: The topper has the highest average.")
        print("The boxplot compares marks spread across three subjects.")

def main():
    students = [
        Student("Asha", [80, 85, 90]),
        Student("Ravi", [70, 75, 72]),
        Student("Neha", [90, 92, 88]),
        Student("Om", [60, 65, 62]),
        Student("Sara", [75, 78, 80]),
        Student("Amit", [55, 60, 58]),
        Student("Raj", [82, 80, 84]),
        Student("Mira", [68, 72, 70])
    ]
    ClassAnalyzer(students).analyze()

if __name__ == "__main__":
    main()
```

Expected output / result

```
Topper: Neha
Class average: 74.42
A figure appears with:
- Horizontal bar chart of student averages
- Boxplot for three subjects
Saved as student_analysis.png
Observation: Neha has the highest average. The boxplot compares the spread of marks.
```

Q20. Numerical Integration and Error Analysis

Simple idea: Integrates x^2 from 0 to 1, whose exact integral is 1/3. Both methods become more accurate with more subintervals.

Python code

```
import matplotlib.pyplot as plt

class Integrator:
    def integrate(self, f, a, b, n):
        raise NotImplementedError

class Trapezoid(Integrator):
    def integrate(self, f, a, b, n):
        h = (b - a) / n
        total = (f(a) + f(b)) / 2
        for i in range(1, n):
            total += f(a + i*h)
        return total * h

class Midpoint(Integrator):
    def integrate(self, f, a, b, n):
        h = (b - a) / n
        total = 0
        for i in range(n):
            total += f(a + (i + 0.5)*h)
        return total * h

def main():
    f = lambda x: x*x
    exact = 1/3                # integral of x^2 from 0 to 1
    ns = [4, 8, 16, 32]
    methods = [Trapezoid(), Midpoint()]
    errors = {}

    for method in methods:
        errors[type(method).__name__] = []
        print(type(method).__name__)
        for n in ns:
            result = method.integrate(f, 0, 1, n)
            error = abs(exact - result)
            errors[type(method).__name__].append(error)
            print("n =", n, "Result =", round(result, 6),
                  "Error =", round(error, 6))

    fig, ax = plt.subplots()
    for name, values in errors.items():
        ax.plot(ns, values, marker="o", label=name)
    ax.set_title("Integration Error")
    ax.set_xlabel("Number of subintervals")
    ax.set_ylabel("Absolute error")
    ax.legend()
    fig.savefig("integration_error.png")
    plt.show()
    print("Observation: Error generally decreases as n increases.")

if __name__ == "__main__":
    main()
```

Expected output / result

```
Trapezoid
n = 4  Result = 0.34375  Error = 0.010417
n = 8  Result = 0.335938  Error = 0.002604
n = 16 Result = 0.333984 Error = 0.000651
n = 32 Result = 0.333496 Error = 0.000163
Midpoint
n = 4  Result = 0.328125 Error = 0.005208
n = 8  Result = 0.332031 Error = 0.001302
n = 16 Result = 0.333008 Error = 0.000326
n = 32 Result = 0.333252 Error = 0.000081
A graph appears showing error decreasing as n increases.
```